

A RANDOM WALK THROUGH RISK

Gambler's Ruin

Four betting scenarios — and the tactics that crack them

From a biased coin → closed-form formulas, clever code, a little linear algebra

4 scenarios • 1 underlying random walk



Why should an ML team care about a gambler?

Every system below is the **same mathematical object** in a different costume: a process that hops between states, where the next step depends only on where you are *now*.

Reinforcement Learning

An agent choosing actions *is* a gambler choosing bets. Value iteration solves it.

Google PageRank

A surfer clicking links at random; page rank = the stationary distribution.

Reliability & Networks

Time-to-failure, churn, how fast something spreads = a first-hitting time.

Quant Finance

Stock prices, ruin probability, risk of bankruptcy.

Big idea: learn it once on a toy gambling problem, and a huge slice of modern ML comes for free.

One coin, one number line

Every scenario is the **same picture**: a token on a number line that steps **right (prob. p)** or **left (prob. $q = 1 - p$)** each round. All that changes is the rule at the edges — and the question we ask.



Why it's solvable: the Markov property

Where you go next depends *only* on where you stand now — never on how you got there. Memoryless.

The one trick we reuse everywhere

Condition on the very first step:

$$f(\text{here}) = p f(\text{step right}) + q f(\text{step left}) (+ \text{cost})$$

Four scenarios

Same random walk, four disguises. We'll state each puzzle, then crack it — watching *which* tactic the problem rewards.

1 The Classic Gambler

Fixed \$1 bets. Her odds, her ruin, her time at the table.

2 The Bold Gambler

All-in betting — cracked by reading binary digits.

3 The Cursed Gambler

A glass ceiling that bounces her back down.

4 The Stock Market

A drifting price: long-run behaviour & first hits.

One idea, four costumes — and a different tool for each.

01

SCENARIO 1

The Classic Gambler

A biased coin, a target fortune, and one wall she never wants to hit.

The setup

She walks in with $\$k$. Every round she stakes $\$1$ on a biased coin: heads (prob. p) and she's up a dollar; tails (prob. q) and she's down one. She stands up the instant she hits her **target $\$N$** — or busts out at **$\0** .

Three things we want to know

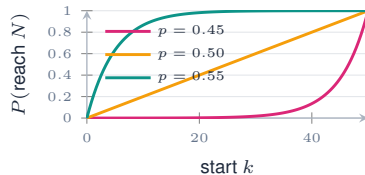
1. What's her chance of reaching $\$N$ before going broke?
2. If she ditches the target and just plays until ruin — can she get *infinitely* rich?
3. How long will she sit at the table before it's over?

She starts somewhere between the two walls — and the walls decide her fate.

Cracking it — her odds of winning

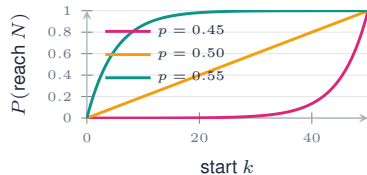
1. **Condition on the first bet.** From $\$k$ she lands on $\$k+1$ (prob. p) or $\$k-1$ (prob. q):

$$T(k) = pT(k+1) + qT(k-1).$$



It all rides on q/p . A closed form is the dream:
instant, exact, no computer needed.

Cracking it — her odds of winning



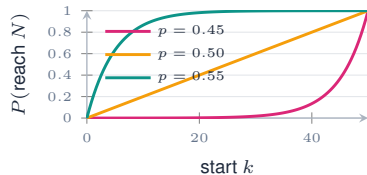
1. **Condition on the first bet.** From $\$k$ she lands on $\$k+1$ (prob. p) or $\$k-1$ (prob. q):

$$T(k) = pT(k+1) + qT(k-1).$$

2. **Anchor the ends:** $T(0) = 0$, $T(N) = 1$.

It all rides on q/p . A closed form is the dream:
instant, exact, no computer needed.

Cracking it — her odds of winning



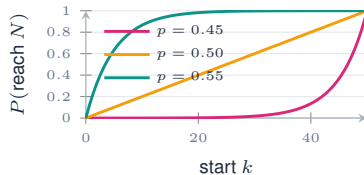
1. **Condition on the first bet.** From $\$k$ she lands on $\$k+1$ (prob. p) or $\$k-1$ (prob. q):

$$T(k) = pT(k+1) + qT(k-1).$$

2. **Anchor the ends:** $T(0) = 0$, $T(N) = 1$.
3. **Solve it.** Guess $T(k) = x^k \Rightarrow px^2 - x + q = 0$, roots $x = 1$ and $x = q/p$.

It all rides on q/p . A closed form is the dream:
instant, exact, no computer needed.

Cracking it — her odds of winning



1. **Condition on the first bet.** From $\$k$ she lands on $\$k+1$ (prob. p) or $\$k-1$ (prob. q):

$$T(k) = pT(k+1) + qT(k-1).$$

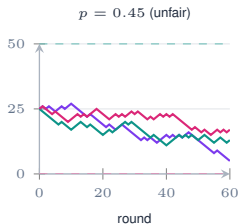
2. **Anchor the ends:** $T(0) = 0$, $T(N) = 1$.
3. **Solve it.** Guess $T(k) = x^k \Rightarrow px^2 - x + q = 0$, roots $x = 1$ and $x = q/p$.
4. **Fit the ends to the two roots:**

$$p \neq q : T(k) = \frac{1 - (q/p)^k}{1 - (q/p)^N}; \quad p = q : T(k) = \frac{k}{N}.$$

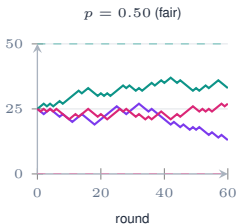
It all rides on q/p . A closed form is the dream:
instant, exact, no computer needed.

Six gamblers, three coins

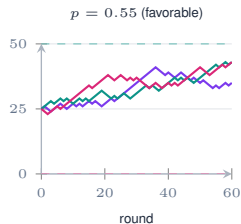
Each line is one simulated life: start at wealth 25, walls at 0 and 50. Same rules, wildly different fates.



slight disadvantage drags everyone to ruin



a coin-flip of destinies



a slight edge, most paths climb

Cracking it — forever, and for how long?

Can she get infinitely rich?

Let $N \rightarrow \infty$.

Fair or against her ($p \leq \frac{1}{2}$): ruin is **certain**, $T \rightarrow 0$.

Only with a true edge ($p > \frac{1}{2}$): $T \rightarrow 1 - (q/p)^k > 0$.

This is “risk of ruin” — why the house, on its sliver of edge, takes everyone eventually.

How long at the table?

Same trick, +1 for the round just played:

$$E_i = p E_{i+1} + q E_{i-1} + 1, \quad E_0 = E_N = 0.$$

The fair game collapses to a gem:

$$E_i = i (N - i).$$

Longest right in the middle.

Predict, then click

Fair coin, start \$50, target \$100 — how many rounds do you expect?

Cracking it — forever, and for how long?

Can she get infinitely rich?

Let $N \rightarrow \infty$.

Fair or against her ($p \leq \frac{1}{2}$): ruin is **certain**, $T \rightarrow 0$.

Only with a true edge ($p > \frac{1}{2}$): $T \rightarrow 1 - (q/p)^k > 0$.

This is “risk of ruin” — why the house, on its sliver of edge, takes everyone eventually.

How long at the table?

Same trick, +1 for the round just played:

$$E_i = p E_{i+1} + q E_{i-1} + 1, \quad E_0 = E_N = 0.$$

The fair game collapses to a gem:

$$E_i = i (N - i).$$

Longest right in the middle.

Predict, then click

Fair coin, start \$50, target \$100 — how many rounds do you expect?

$$E_{50} = 50 \times 50 = 2500 \text{ rounds.}$$

Where this bites in real life: position sizing, insurance reserves, bankroll management.

02

SCENARIO 2

The Bold Gambler

Tired of nickel-and-dime bets, she starts going all in.

The setup

New attitude, new rule. With **less than half** the target ($k < \frac{N}{2}$) she shoves **all** her chips — double up (prob. p) or bust (prob. q). With **at least half** ($k \geq \frac{N}{2}$) she bets exactly enough to finish, $N - k$: hit the target (prob. p), or slip back to $2k - N$ (prob. q) and keep going.

The question

With this bold rule, what are her odds of winning — and how long does the game run?

Secret weapon

Write the fraction k/N in **binary**. The whole strategy turns into reading its digits.

Cracking it — think in binary

1. **Recurrence by region.** $P(x) = p P(2x)$ for $0 < x < \frac{N}{2}$; $P(x) = q P(2x-N) + p$ for $\frac{N}{2} \leq x < N$; ends $P(0) = 0, P(N) = 1$.

Cracking it — think in binary

1. **Recurrence by region.** $P(x) = p P(2x)$ for $0 < x < \frac{N}{2}$; $P(x) = q P(2x-N) + p$ for $\frac{N}{2} \leq x < N$; ends $P(0) = 0, P(N) = 1$.
2. **Spot the binary.** Doubling is a *left shift* of the digits of k/N ; losing from the top shifts *and drops* the leading 1. So the game just reads $k/N = 0.b_1b_2b_3 \dots$ bit by bit.

Cracking it — think in binary

1. **Recurrence by region.** $P(x) = p P(2x)$ for $0 < x < \frac{N}{2}$; $P(x) = q P(2x-N) + p$ for $\frac{N}{2} \leq x < N$; ends $P(0) = 0, P(N) = 1$.
2. **Spot the binary.** Doubling is a *left shift* of the digits of k/N ; losing from the top shifts *and drops* the leading 1. So the game just reads $k/N = 0.b_1b_2b_3 \dots$ bit by bit.
3. **Walk the bits**, carrying a running probability: bit 0 $\Rightarrow \times p$; bit 1 $\Rightarrow \times q, + p$.

Cracking it — think in binary

1. **Recurrence by region.** $P(x) = p P(2x)$ for $0 < x < \frac{N}{2}$; $P(x) = q P(2x-N) + p$ for $\frac{N}{2} \leq x < N$; ends $P(0) = 0, P(N) = 1$.
2. **Spot the binary.** Doubling is a *left shift* of the digits of k/N ; losing from the top shifts *and drops* the leading 1. So the game just reads $k/N = 0.b_1b_2b_3 \dots$ bit by bit.
3. **Walk the bits**, carrying a running probability: bit 0 $\Rightarrow \times p$; bit 1 $\Rightarrow \times q, + p$.
4. **Duration is the same walk:** bit 0 $\Rightarrow 1 + p E$; bit 1 $\Rightarrow 1 + q E$.

Cracking it — think in binary

1. **Recurrence by region.** $P(x) = p P(2x)$ for $0 < x < \frac{N}{2}$; $P(x) = q P(2x-N) + p$ for $\frac{N}{2} \leq x < N$; ends $P(0) = 0, P(N) = 1$.
2. **Spot the binary.** Doubling is a *left shift* of the digits of k/N ; losing from the top shifts *and drops* the leading 1. So the game just reads $k/N = 0.b_1b_2b_3 \dots$ bit by bit.
3. **Walk the bits**, carrying a running probability: bit 0 $\Rightarrow \times p$; bit 1 $\Rightarrow \times q, + p$.
4. **Duration is the same walk:** bit 0 $\Rightarrow 1 + p E$; bit 1 $\Rightarrow 1 + q E$.

Cracking it — think in binary

1. **Recurrence by region.** $P(x) = p P(2x)$ for $0 < x < \frac{N}{2}$; $P(x) = q P(2x-N) + p$ for $\frac{N}{2} \leq x < N$; ends $P(0) = 0, P(N) = 1$.
2. **Spot the binary.** Doubling is a *left shift* of the digits of k/N ; losing from the top shifts *and drops* the leading 1. So the game just reads $k/N = 0.b_1b_2b_3 \dots$ bit by bit.
3. **Walk the bits**, carrying a running probability: bit 0 $\Rightarrow \times p$; bit 1 $\Rightarrow \times q, + p$.
4. **Duration is the same walk:** bit 0 $\Rightarrow 1 + p E$; bit 1 $\Rightarrow 1 + q E$.

How to solve it

No tidy formula here — but the recursion is a **five-line loop** over a binary string. Sometimes the right move is just to *code the recurrence*.

The takeaway

Pick the right **representation** (binary) and a hard problem dissolves. And under bad odds, **bold beats cautious** — provably.

03

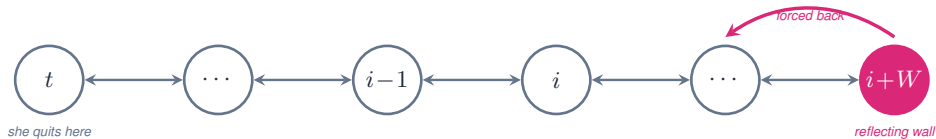
SCENARIO 3

The Cursed Gambler

A glass ceiling she can climb to — but never break through.

The setup

A bad streak breeds superstition. She's convinced she's cursed: once her wealth reaches some level, she can rise at most W above it. Touch that ceiling and the next round is a guaranteed loss — she's **bounced straight back down** (a reflecting wall). Rattled, she vows to quit the moment her wealth sinks to t .



How many rounds will she play before quitting at t (starting from k)?

We'll want the answer exactly — as a fraction — so we keep it under a big prime.

Cracking it — one drop at a time

1. **Slice the fall.** Dropping from k to t is just dropping one level, then the next, then the next. By linearity of expectation, *add the costs*:

$$E(k) = \sum_{j=t+1}^k E(\text{fall from } j \text{ to } j-1).$$

Cracking it — one drop at a time

1. **Slice the fall.** Dropping from k to t is just dropping one level, then the next, then the next. By linearity of expectation, *add the costs*:

$$E(k) = \sum_{j=t+1}^k E(\text{fall from } j \text{ to } j-1).$$

2. **Spot the repetition.** Every single-level drop sees the *same* local picture — a width- W window with the reflecting wall on top — so each costs the same constant X .

Cracking it — one drop at a time

1. **Slice the fall.** Dropping from k to t is just dropping one level, then the next, then the next. By linearity of expectation, *add the costs*:

$$E(k) = \sum_{j=t+1}^k E(\text{fall from } j \text{ to } j-1).$$

2. **Spot the repetition.** Every single-level drop sees the *same* local picture — a width- W window with the reflecting wall on top — so each costs the same constant X .
3. **Multiply:** $E(k) = X(k - t)$. One hard quantity, then arithmetic.

Cracking it — one drop at a time

1. **Slice the fall.** Dropping from k to t is just dropping one level, then the next, then the next. By linearity of expectation, *add the costs*:

$$E(k) = \sum_{j=t+1}^k E(\text{fall from } j \text{ to } j-1).$$

2. **Spot the repetition.** Every single-level drop sees the *same* local picture — a width- W window with the reflecting wall on top — so each costs the same constant X .
3. **Multiply:** $E(k) = X(k - t)$. One hard quantity, then arithmetic.
4. **Find X** from the local chain $E(x) = p E(x+1) + q E(x-1) + 1$, with the wall giving $E(\text{top}) = E(\text{top}-1) + 1$. Back-substitute downward — a short $W-1$ step loop gives X .

Cracking it — one drop at a time

1. **Slice the fall.** Dropping from k to t is just dropping one level, then the next, then the next. By linearity of expectation, *add the costs*:

$$E(k) = \sum_{j=t+1}^k E(\text{fall from } j \text{ to } j-1).$$

2. **Spot the repetition.** Every single-level drop sees the *same* local picture — a width- W window with the reflecting wall on top — so each costs the same constant X .
3. **Multiply:** $E(k) = X(k - t)$. One hard quantity, then arithmetic.
4. **Find X** from the local chain $E(x) = p E(x+1) + q E(x-1) + 1$, with the wall giving $E(\text{top}) = E(\text{top}-1) + 1$. Back-substitute downward — a short $W-1$ step loop gives X .

Cracking it — one drop at a time

1. **Slice the fall.** Dropping from k to t is just dropping one level, then the next, then the next. By linearity of expectation, *add the costs*:

$$E(k) = \sum_{j=t+1}^k E(\text{fall from } j \text{ to } j-1).$$

2. **Spot the repetition.** Every single-level drop sees the *same* local picture — a width- W window with the reflecting wall on top — so each costs the same constant X .
3. **Multiply:** $E(k) = X(k - t)$. One hard quantity, then arithmetic.
4. **Find X** from the local chain $E(x) = p E(x+1) + q E(x-1) + 1$, with the wall giving $E(\text{top}) = E(\text{top}-1) + 1$. Back-substitute downward — a short $W-1$ step loop gives X .

Keep it exact

X is a fraction a/b . Floating point would quietly corrupt it; a **modular inverse** keeps it perfect — output $a b^{-1} \bmod (10^9+7)$. The same move reappears all over competitive programming.

04

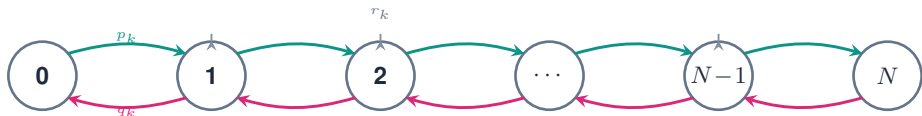
SCENARIO 4

The Stock Market

Same random walk, new costume — a drifting price.

The setup

Step away from the casino and watch a stock. Each day the price ticks **up one** (prob. p_k), **down one** (prob. q_k), or **holds** (prob. r_k) — rates that can depend on the current price k . It lives on $\{0, 1, \dots, N\}$ and can't escape the ends.



Two questions

1. Long-run: how much time does the price spend at each level, and what's the *average* price?
2. From price a , how long until it first touches price b ?

Cracking it — the long-run picture



1. Long-run proportions π are unchanged by one more step:
 $\pi = \pi P$.

Where it spends its time — and the start point is forgotten.

Closed-form ratios, then one normalisation — a few lines of cumulative products.

Cracking it — the long-run picture



1. Long-run proportions π are unchanged by one more step:
 $\pi = \pi P$.
2. **Balance neighbours:** up-flow = down-flow,

$$\pi_{i-1}p_{i-1} = \pi_i q_i \Rightarrow \pi_i = \pi_{i-1} \frac{p_{i-1}}{q_i}.$$

Where it spends its time — and the start point is forgotten.

Closed-form ratios, then one normalisation — a few lines of cumulative products.

Cracking it — the long-run picture



1. Long-run proportions π are unchanged by one more step:
 $\pi = \pi P$.

2. **Balance neighbours:** up-flow = down-flow,

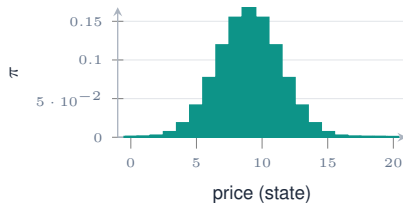
$$\pi_{i-1}p_{i-1} = \pi_i q_i \Rightarrow \pi_i = \pi_{i-1} \frac{p_{i-1}}{q_i}.$$

3. Chain the ratios from π_0 ; pin π_0 with $\sum_i \pi_i = 1$.

Where it spends its time — and the start point is forgotten.

Closed-form ratios, then one normalisation — a few lines of cumulative products.

Cracking it — the long-run picture



1. Long-run proportions π are unchanged by one more step:
 $\pi = \pi P$.

2. **Balance neighbours:** up-flow = down-flow,

$$\pi_{i-1}p_{i-1} = \pi_i q_i \Rightarrow \pi_i = \pi_{i-1} \frac{p_{i-1}}{q_i}.$$

3. Chain the ratios from π_0 ; pin π_0 with $\sum_i \pi_i = 1$.
4. Average price = $\sum_i i \pi_i$.

Where it spends its time — and the start point is forgotten.

Closed-form ratios, then one normalisation — a few lines of cumulative products.

Cracking it — time to hit a target price

1. Let E_x be the expected days from x to b . Condition on the first move:

$$E_x = p_x E_{x+1} + r_x E_x + q_x E_{x-1} + 1, \quad E_b = 0.$$

Cracking it — time to hit a target price

1. Let E_x be the expected days from x to b . Condition on the first move:

$$E_x = p_x E_{x+1} + r_x E_x + q_x E_{x-1} + 1, \quad E_b = 0.$$

2. One equation per state — stack them: $A \mathbf{E} = \mathbf{1}$. The matrix A is tridiagonal, *except* row b , pinned by $E_b = 0$.

Cracking it — time to hit a target price

1. Let E_x be the expected days from x to b . Condition on the first move:

$$E_x = p_x E_{x+1} + r_x E_x + q_x E_{x-1} + 1, \quad E_b = 0.$$

2. One equation per state — stack them: $A \mathbf{E} = \mathbf{1}$. The matrix A is tridiagonal, *except* row b , pinned by $E_b = 0$.
3. The rates vary, so there's no clean formula — recognise a **linear system** and let a solver finish it.

Cracking it — time to hit a target price

1. Let E_x be the expected days from x to b . Condition on the first move:

$$E_x = p_x E_{x+1} + r_x E_x + q_x E_{x-1} + 1, \quad E_b = 0.$$

2. One equation per state — stack them: $A \mathbf{E} = \mathbf{1}$. The matrix A is tridiagonal, *except* row b , pinned by $E_b = 0$.
3. The rates vary, so there's no clean formula — recognise a **linear system** and let a solver finish it.

Cracking it — time to hit a target price

1. Let E_x be the expected days from x to b . Condition on the first move:

$$E_x = p_x E_{x+1} + r_x E_x + q_x E_{x-1} + 1, \quad E_b = 0.$$

2. One equation per state — stack them: $A \mathbf{E} = \mathbf{1}$. The matrix A is tridiagonal, *except* row b , pinned by $E_b = 0$.
3. The rates vary, so there's no clean formula — recognise a **linear system** and let a solver finish it.

*Real-world: that stationary π is **PageRank**; a first-hitting time is expected **time-to-failure**, **time-to-churn**, or how long something takes to spread.*

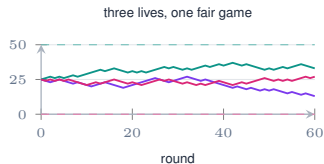
Four ways to crack a recurrence

Notice we never used one hammer for everything. The puzzle tells you which tool is cheapest.

- 1 **Closed form** (*Scenario 1*) — constant coefficients → characteristic equation → a formula. Instant, exact, hands you insight (that q/p ratio).
- 2 **Walk the recursion** (*Scenario 2*) — formula's ugly, but the states are structured → a tiny loop.
- 3 **Decompose** (*Scenario 3*) — symmetry collapses it to one repeated piece → algebra, then a short loop.
- 4 **Linear system** (*Scenario 4*) — coefficients vary → "it's just $A\mathbf{E} = \mathbf{b}$," hand it to a solver.

Always in your back pocket

Simulate. Stuck? Run the random walk thousands of times and average. Approximate — but it never fails to apply.

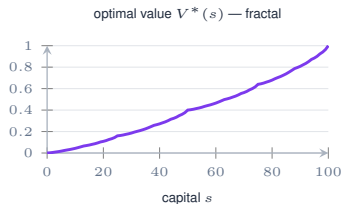


Reach for the cheapest tool that works — and know when to stop hunting for a formula.

Beyond the casino

Strip off the story and this same random walk is everywhere.

- **Risk of ruin** — position sizing, insurance reserves and bankroll math are exactly Scenario 1's ruin probability.
- **Learning the best bet** — let the gambler *choose* her stake and you've built reinforcement learning; the optimal play is bold, with a famously fractal value curve (right).
- **Reliability & ops** — a first-hitting time is expected *time-to-failure*, time-to-churn, or how fast a rumour, infection, or queue backlog spreads.
- **PageRank** — the stationary distribution of a random web-surfer; the next slide unpacks it.

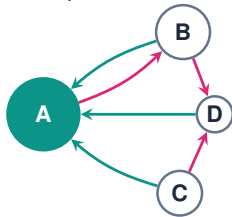


The same machinery quietly powers a lot of modern ML.

PageRank — the random surfer

Swap the number line for a **graph of web pages**. A surfer stands on a page and, each step, clicks a *random* outgoing link. Run this forever: the fraction of time spent on each page *is* its rank. Same stationary-distribution math as the drifting stock — just on a messier state space.

1. **Same balance condition.** The rank vector is fixed under one more click: $\pi = \pi M$, where M_{ij} is the chance of hopping $i \rightarrow j$.
2. **Vote by linking.** A page is important if important pages link to it — importance flows along edges, split evenly across a page's out-links.
3. **The teleport fix.** A surfer can get stuck on dead-end or cyclic pages. With prob. $1-d$ (≈ 0.15) she **teleports** to a random page. That damping makes the chain *irreducible & aperiodic* — so a *unique* π exists and is reached from any start.
4. **Solve by iterating.** Start uniform, apply M repeatedly — power iteration converges fast on a web-scale, billion-node graph.



A is biggest: three pages point at it. Node size $\propto$ rank.

Why it mattered

Treating links as votes — and ranking by the random walk's stationary distribution — is the idea that launched Google. The same eigenvector trick now drives citation ranking, recommendation graphs, and fraud-ring detection.

Let's talk

This deck is built to **pause at each scenario** — predict the next step before the click. A few things worth arguing about:

- Even at fair odds, distance-to-target crushes you — why does that feel so counterintuitive?
- When is hunting for a closed form a waste of time versus just coding the recurrence?
- Where have you met a “reflecting wall” — a hard cap that reshapes a system?
- Bold play beating cautious play under bad odds — does that match your gut on risk?
- “Where does this spend most of its time?” — where else is that the useful question?
- What quantity in your world is secretly a *first hitting time*?

THANK YOU

One coin. Four scenarios. A handful of tactics.

Strip away the story and it's the same random walk every time — cracked with whatever's cheapest: a formula, a few lines of code, or a quick simulation.

GO DEEPER

Grinstead & Snell — *Introduction to Probability*, Ch. 12 (Random Walks)

J. R. Norris — *Markov Chains* (stationary distributions & hitting times)

Sutton & Barto — *Reinforcement Learning* (the Gambler's Problem)

Questions? Let's talk random walks.